

Aleksandra Szczypawka

Zadanie 3 - Koncepcja

Napisanie programu tworzącego odpowiednie procesy na osobnych wątkach wykonujących różne typy operacji na wspólnym buforze liczbowym.

- Procesy A1 generują kolejne liczby parzyste modulo 50, jeżeli w buforze jest mniej niż 10 liczb parzystych.
 - Procesy A2 generują kolejne liczby nieparzyste modulo 50, jeżeli liczb parzystych w buforze jest więcej niż nieparzystych.
 - Procesy B1 zjadają liczby parzyste pod warunkiem, że bufor zawiera co najmniej 3 liczby.
 - Procesy B2 zjadają liczby nieparzyste, pod warunkiem, że bufor zawiera co najmniej 7 liczb.
- Synchronizacja ma być zaimplementowana z wykorzystaniem 5-ciu semaforów binarnych.

Zadanie realizowane w Pythonie

1. Implementacja klasy *BinarySemaphore* :

```
class BinarySemaphore:
    def __init__(self, state):
        self._state = state # 1 - dostępny, 0 - zajęty

    def P(self):
        while self._state == 0:
            pass
        self._state = 0

    def V(self):
        self._state = 1
```

2. Zainicjalizowanie bufora, semafora głównego, semaforów procesów (typów A1, A2, B1, B2) i liczników oczekujących procesów

```
buffer = deque
mutex = Semaphore(1)
prod_odd_mod_50_mutex = Semaphore(0)
prod_even_mod_50_mutex = Semaphore(0)
cons_even_mutex = Semaphore(0)
cons_odd_mutex = Semaphore(0)

num_of_prod_even_mod_50_waiting = 0
num_of_prod_odd_mod_50_waiting = 0
num_of_cons_even_waiting = 0
num_of_cons_odd_waiting = 0
```

3. Zdefiniowanie warunków produkcji/ konsumpcji dla procesów

```

def can_prod_even_mod_50():
    even_count = sum(1 for x in buffer if x % 2 == 0)
    return even_count < 10

def can_prod_odd_mod_50():
    even_count = sum(1 for x in buffer if x % 2 == 0)
    odd_count = sum(1 for x in buffer if x % 2 != 0)
    return even_count > odd_count

def can_cons_even():
    return len(buffer) >= 3

def can_cons_odd():
    return len(buffer) >= 7

```

4. Zdefiniowanie funkcji, które będą realizowane przez poszczególne typy procesów

```

def prod_odd_mod_50():
    i = 1
    while (1):
        mutex.P()
        if not can_prod_odd_mod_50():
            global num_of_prod_odd_mod_50_waiting
            num_of_prod_odd_mod_50_waiting += 1
            mutex.V()
            prod_odd_mod_50_mutex.P()
            num_of_prod_odd_mod_50_waiting -= 1

        # production/consumption:
        buffer.append(i)
        i += 2
        i = i % 50

        if num_of_cons_even_waiting > 0 and can_cons_even():
            cons_even_mutex.V()
        elif num_of_cons_odd_waiting > 0 and can_cons_odd():
            cons_odd_mutex.V()
        elif num_of_prod_even_mod_50_waiting > 0 and can_prod_even_mod_50():
            prod_even_mod_50_mutex.V()
        else: mutex.V()

```

Funkcja `prod_odd_mod_50`:

- Zajmuje semafor główny, czyli będzie wykonywana, jeśli *mutex* jest wolny, w przeciwnym wypadku zostanie zawieszona.
- Jeśli nie spełniony jest warunek produkcji, to zwiększa licznik odpowiedniego typu procesów, opuszcza semafor główny i zajmuje semafor konkretnego procesu, gdzie jest zawieszany (ponieważ zaczynamy z semaforem w stanie 0).
- Jeśli spełniony jest warunek produkcji (lub zostanie odblokowany przez inną funkcję, gdy spełniony będzie ten warunek) to wykonuje swoją produkcję. W tym przykładzie zapisywanie do bufora kolejnej liczby nieparzystej modulo 50.
- Jeśli oczekuje proces jakiegoś typu i spełnione są warunki uruchomienia go, to zostaje on odwieszony. W przeciwnym przypadku zwalniany jest semafor główny.

Funkcje `prod_even_mod_50`, `cons_even`, `cons_odd` składają się z analogicznych mechanizmów dostępów, różnią się produkcją (konsumpcją) elementów.

5. Stworzenie wątków realizujących określone funkcje (np. `prod_odd_mod_50`) i uruchomienie przypadków testowych, na pustym oraz pełnym buforze.